

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 5

Formularios, validación y gestión del estado

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Formularios, validación y estado local**DURACIÓN** 1 sesión**CAPÍTULO** Cap. 3 — Gestión del estado e interactividad**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guías de Laboratorio N.º 1 a N.º 4**NOTA · Continúa desde la Guía N.º 4**

Ya sabes mostrar colecciones de datos. En esta práctica el usuario los **creará**: construirás un formulario con `TextFormField`, lo **validarás** con `Form`, y con `setState` agregarás cada ítem a una lista que se actualiza en pantalla.

01 Objetivos

Objetivo general. Capturar y validar la entrada del usuario mediante formularios, y gestionar estado local para reflejar los cambios en la interfaz, construyendo una pequeña app de tareas (to-do).

Objetivos específicos:

- Capturar texto con `TextField` y `TextEditingController`.
- Construir un formulario validado con `Form`, `TextFormField` y `GlobalKey<FormState>`.
- Actualizar una lista en memoria con `setState` y reflejarla con `ListView`.
- Manejar el ciclo de vida del controlador (`dispose`) para liberar recursos.
- Dar retorno visual al usuario (por ejemplo, un `SnackBar`) tras una acción válida.

02 Marco teórico

DEFINICIÓN · TextField y TextEditingController

`TextField` es el campo de entrada de texto. Un `TextEditingController` se asocia al campo para leer o modificar su contenido desde el código.

DEFINICIÓN · Form, TextFormField y validación

`Form` agrupa campos `TextFormField`, cada uno con una función `validator` que devuelve un mensaje de error o `null` si el valor es válido. Una `GlobalKey<FormState>` permite disparar la validación de todo el formulario con `.validate()`.

DEFINICIÓN · Estado local y dispose

El estado que vive dentro de un `StatefulWidget` se modifica con `setState`. Los controladores deben liberarse en `dispose` para evitar fugas de memoria.

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 a N.º 4 (entorno operativo, widgets, listas y navegación).

Conocimientos de apoyo:

StatefulWidget y setState

ListView

Funciones en Dart

NOTA · Estado local vs. Provider

En esta guía manejamos el estado dentro de un solo widget (`setState`). Cuando el estado deba compartirse entre muchas pantallas, en el Cap. 4 pasaremos a **Provider**. Empezar aquí facilita entender ese salto.

04 Desarrollo · Parte A — Captura de texto

1 Preparar la pantalla con estado

1. Crea un proyecto llamado `tareas_app`. En `lib/main.dart`, arma un `StatefulWidget` con un controlador y la lista de tareas:

LIB/MAIN.DART · DART

```
class PantallaTareas extends StatefulWidget {
  const PantallaTareas({super.key});

  @override
  State<PantallaTareas> createState() => _PantallaTareasState();
}

class _PantallaTareasState extends State<PantallaTareas> {
  final _controller = TextEditingController();
  final List<String> _tareas = [];

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
  // ... build() en el siguiente paso
}
```

ADVERTENCIA · No olvides dispose

Cada `TextEditingController` reserva recursos. Liberarlo en `dispose` evita fugas de memoria cuando la pantalla se destruye.

2 Agregar el campo de texto y un botón

1. En el `build`, combina un `TextField` con un botón. Al pulsar, el texto se agrega a la lista con `setState`:

LIB/MAIN.DART · DART

```
void _agregar() {
  final texto = _controller.text.trim();
  if (texto.isEmpty) return;
  setState(() {
    _tarefas.add(texto);
    _controller.clear();
  });
}
```

DART · DENTRO DEL BUILD

```
Row(
  children: [
    Expanded(
      child: TextField(
        controller: _controller,
        decoration: const InputDecoration(
          labelText: 'Nueva tarea',
          border: OutlineInputBorder(),
        ),
        onSubmitted: (_) => _agregar(),
      ),
    ),
    const SizedBox(width: 8),
    IconButton(
      icon: const Icon(Icons.add),
      onPressed: _agregar,
    ),
  ],
)
```

NOTA · Expanded en una Row

`Expanded` hace que el `TextField` ocupe todo el ancho disponible y deje sitio al botón. Sin él, el campo no sabría qué ancho tomar dentro de la `Row`.

3 Mostrar la lista y permitir borrar

1. Debajo del campo, muestra las tareas con un `ListView.builder` envuelto en `Expanded`, con un icono para eliminar cada una:

DART · DENTRO DEL BUILD

```
Expanded(
  child: ListView.builder(
    itemCount: _tareas.length,
    itemBuilder: (context, index) {
      return ListTile(
        title: Text(_tareas[index]),
        trailing: IconButton(
          icon: const Icon(Icons.delete),
          onPressed: () => setState(
            () => _tareas.removeAt(index),
          ),
        ),
      );
    },
  ),
);
```

2. Ejecuta: escribe una tarea, pulsa el botón y observa cómo aparece en la lista; toca el basurero para quitarla.

05 Desarrollo · Parte B — Formulario validado

4 Envolver los campos en un Form

1. Para validar la entrada, usa un `Form` con una `GlobalKey<FormState>`. Declárala en el State:

LIB/MAIN.DART · DART

```
final _formKey = GlobalKey<FormState>();
```

2. Reemplaza el `TextField` por un `TextFormField` con un `validator`, dentro de un `Form`:

DART · DENTRO DEL BUILD

```
Form(
  key: _formKey,
  child: TextFormField(
    controller: _controller,
    decoration: const InputDecoration(
      labelText: 'Nueva tarea',
      border: OutlineInputBorder(),
    ),
  ),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return 'Escribe una tarea';
    }
    if (value.trim().length < 3) {
      return 'Debe tener al menos 3 caracteres';
    }
    return null;
  },
)
```

```
    },
  ),
)
```

EJEMPLO · La regla del validator

El `validator` devuelve un **texto de error** para mostrar bajo el campo, o `null` cuando el valor es correcto. Aquí exigimos que no esté vacío y tenga al menos 3 caracteres.

5 Validar antes de agregar

1. Modifica `_agregar` para que solo agregue si el formulario es válido:

```
LIB/MAIN.DART · DART
void _agregar() {
  if (!_formKey.currentState!.validate()) {
    return; // hay errores: no continua
  }
  setState(() {
    _tareas.add(_controller.text.trim());
    _controller.clear();
  });
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Tarea agregada')),
  );
}
```

2. Ejecuta y prueba: si el campo está vacío o es muy corto, aparece el mensaje de error bajo el campo; si es válido, la tarea se agrega y sale un `SnackBar`.

NOTA · `validate()` dispara los validators

`_formKey.currentState!.validate()` ejecuta el `validator` de cada campo del `Form` y devuelve `true` solo si todos pasan. Es el patrón estándar para validar formularios en Flutter.

06 Desarrollo · Parte C — Pulir la interacción

6 Estado vacío y contador

1. Mejora la experiencia mostrando un mensaje cuando no hay tareas y el total en el `AppBar`. Para el estado vacío:

```
DART · PATRÓN CONDICIONAL
_tareas.isEmpty
  ? const Center(child: Text('Sin tareas aun'))
  : Expanded(
    child: ListView.builder(
      // ... itemBuilder como antes
    ),
  ),
```

)

2. En el `AppBar`, muestra el total de tareas en el título:

DART · APPBAR

```
appBar: AppBar(
  title: Text('Tareas (' + _tareas.length.toString() + ')'),
),
```

EJEMPLO · Operador condicional en la UI

La expresión `condicion ? A : B` elige qué widget mostrar según el estado. Es la forma habitual de manejar "vacío vs. con datos" dentro del `build`.

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste la práctica:

- ☐ El campo captura texto mediante `TextEditingController`.
- ☐ El controlador se libera en `dispose`.
- ☐ El validator muestra error si la tarea está vacía o es muy corta.
- ☐ Solo se agrega la tarea cuando el formulario es válido.
- ☐ La lista se actualiza con `setState` al agregar y al borrar.
- ☐ Se muestra el estado vacío y el total de tareas en el `AppBar`.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura del formulario mostrando un mensaje de error de validación; (2) captura de la lista con varias tareas y el contador en el `AppBar`; (3) captura del `SnackBar`; y (4) **reto personal**: agrega un segundo campo (por ejemplo, prioridad con un `DropDownButtonFormField`) y muéstralo en cada `ListTile`. Descríbelo en 3–5 líneas.

08 Cuestionario

1. ¿Cuál es la diferencia entre `TextField` y `TextFormField`?
2. ¿Qué papel cumple la `GlobalKey<FormState>` y el método `validate()`?
3. ¿Qué debe devolver un validator cuando el valor es correcto y cuándo hay error?
4. ¿Por qué es necesario llamar a `dispose` sobre el `TextEditingController`?
5. Explica cómo `setState` logra que la lista se refleje actualizada en la interfaz.

09 Rúbrica de evaluación

Criterio	Puntaje
Captura de texto con controlador y <code>dispose</code>	20 %

Formulario con validación funcionando	30 %
Lista actualizada con setState (agregar/borrar)	25 %
Estado vacío, contador y SnackBar	10 %
Cuestionario resuelto	15 %

10 Referencias

- Flutter — Build a form with validation. docs.flutter.dev/cookbook/forms/validation
- Flutter — Create and style a text field. docs.flutter.dev/cookbook/forms/text-input
- Flutter — Retrieve the value of a text field. docs.flutter.dev/cookbook/forms/retrieve-input
- Flutter — Handle changes to a text field. docs.flutter.dev/cookbook/forms/text-field-changes

Contenido basado en la documentación oficial de Flutter (v3.44). Guía elaborada para INF 662 — UATF.